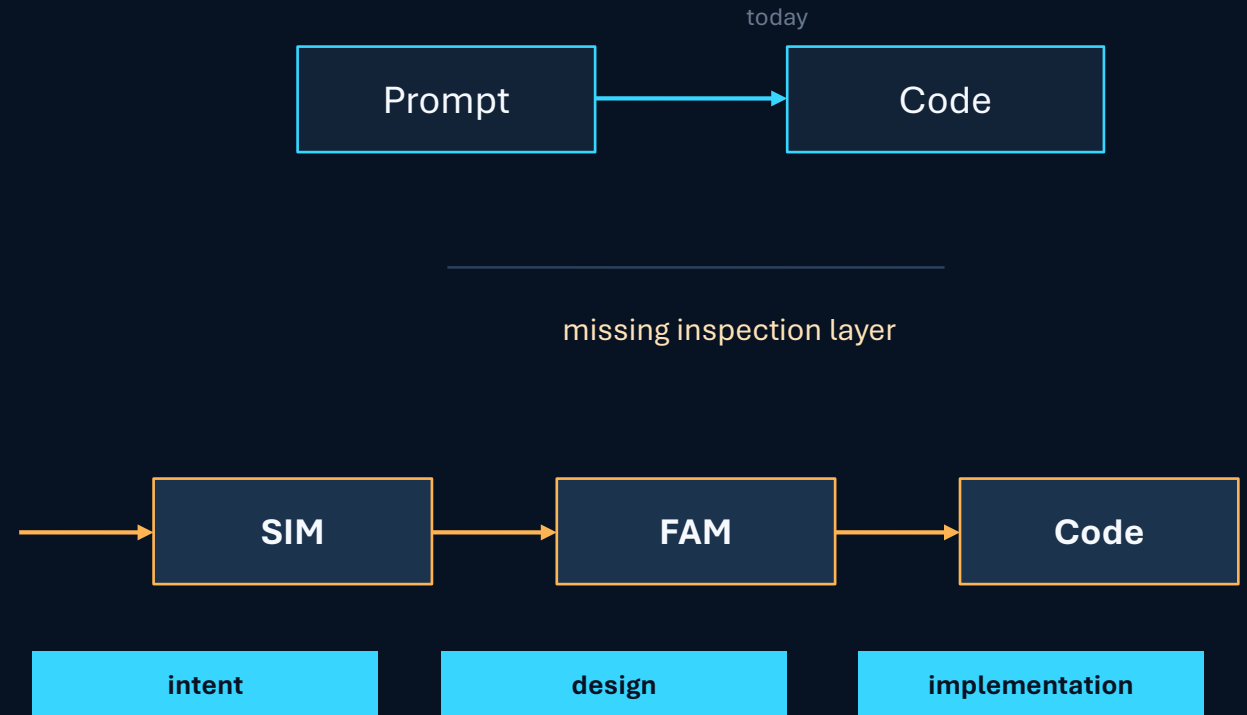


Architect First

Review intent before code exists

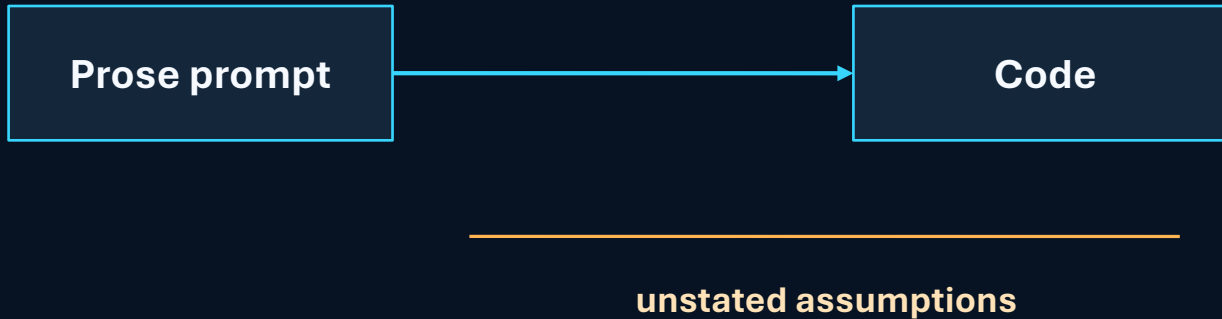
Summary of “Tell Your Coding Agent to Work as an Architect First”



Let the agent declare intent, show the design, pass the gates, then code.

The current workflow hides the dangerous part

Fast code generation is not the same as explicit intent.

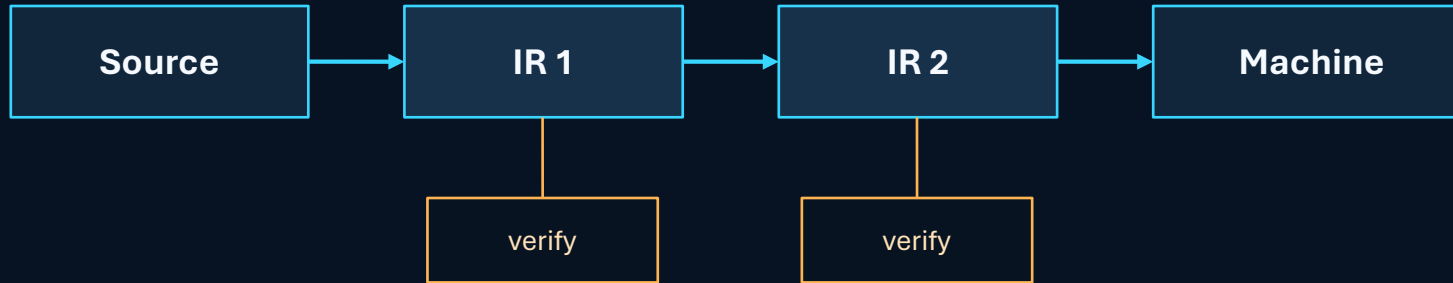


What crashes mid-operation?
Can a critical command vanish silently?
What enforces each guarantee?
How is recovery bounded?

The code may be fluent and tested, but it can still implement the wrong system.

Compiler analogy: missing IRs

Mature compilers do not jump directly from source to machine code.



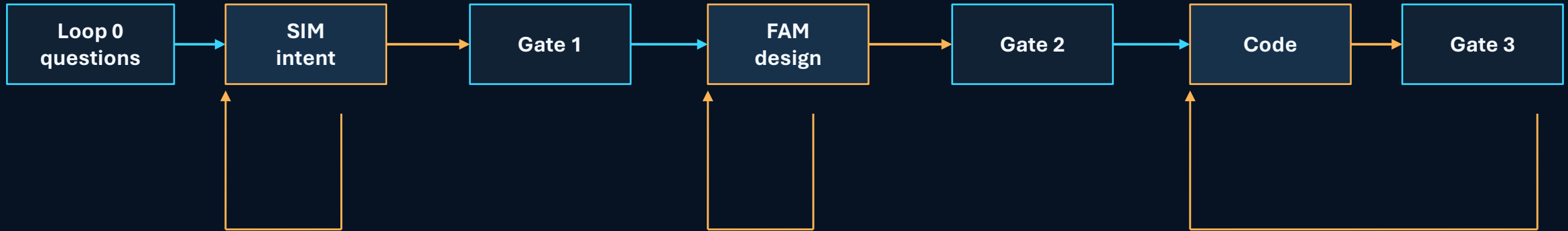
AI coding today often skips that middle layer:



Architect First adds machine-checkable IRs for agentic software construction.

The proposed pipeline: declare, gate, derive, gate, code

Three commitments, three admission gates.

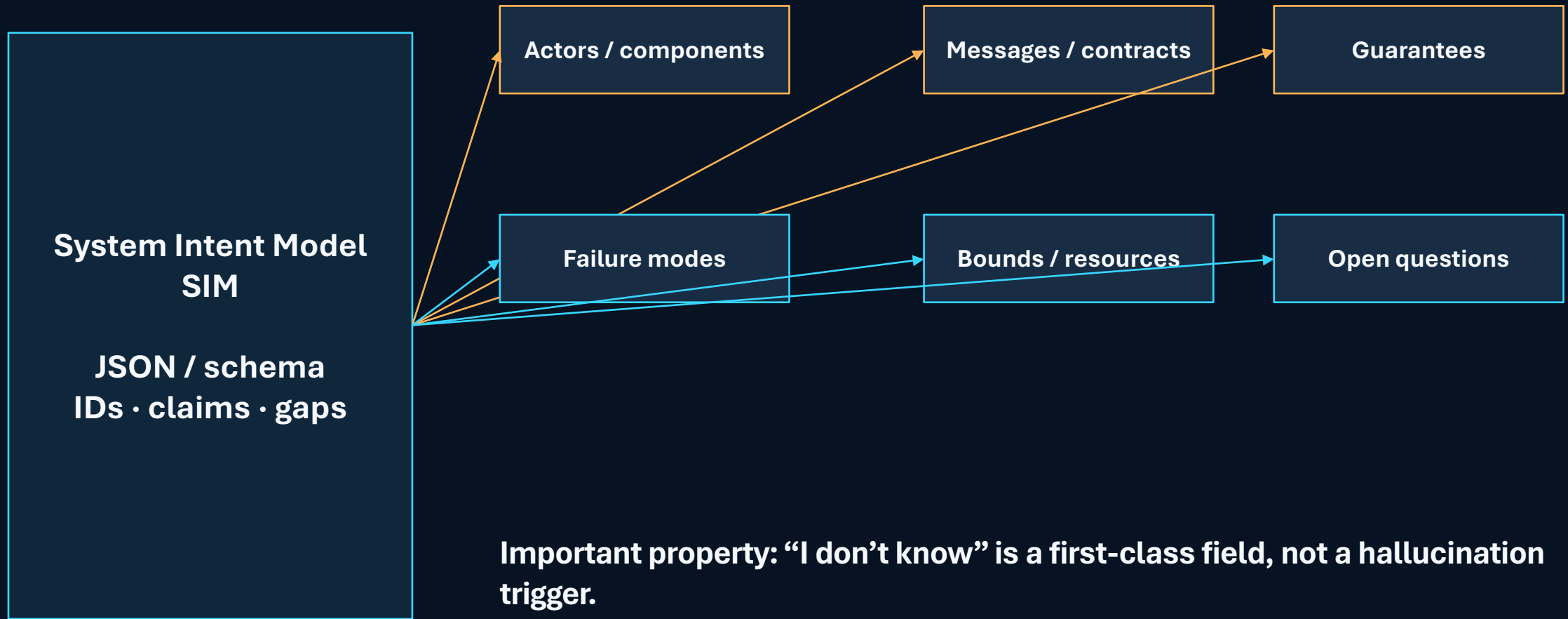


Gate failure is not red ink. It is a structured proof obligation.

PASS moves the artifact forward
BLOCK returns precise findings
ESCALATE or ABORT terminates bounded loops

SIM: system intent as data, not a prose plan

The agent must expose what it thinks the system is.



Gate 1: catches dangerous intent-level omissions

A small, boring checker asks questions real systems force us to answer.

Delivery coherence

Can critical messages vanish silently?

Bounded recovery

Can restarts or retries run forever?

Invariant support

Is a claimed guarantee backed by a mechanism?

Resource limits

Can the system create unbounded instances?

Corrigibility

Can it be stopped without single-point failure?

BLOCK → revise SIM

FAM: architecture as a checkable blueprint

The approved intent is lowered into design.



State × Event table

	Submit	Done	Fail	Shutdown
Idle	✓	✓	✓	✓
Processing	✓	✓	✓	✓
Stopped	✓	✓	✓	✓

Typed messages and ports
State machines and transition tables
Invariants mapped to enforcement locations
Traceability back to SIM IDs

Gate 2: checks the design arithmetic

Once the design exists as data, these become computable facts.

F1
table completeness

F2
block-property match

F3
lowering integrity

F4
port coherence

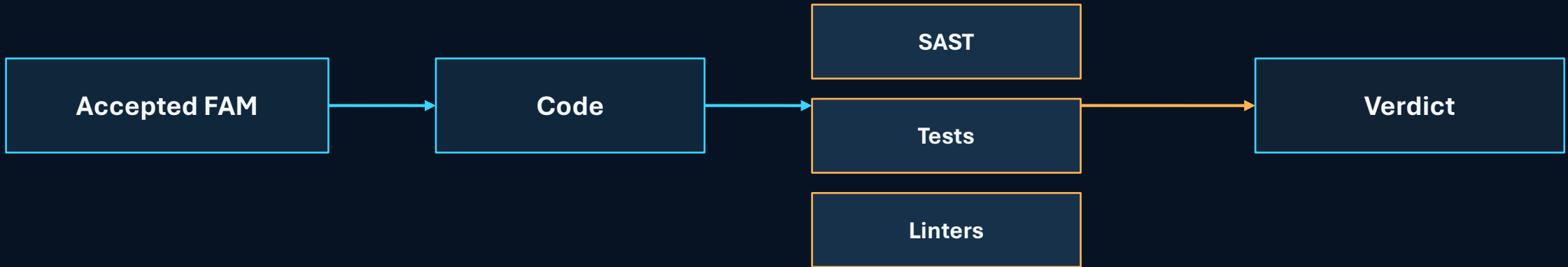
F5
guard-as-code

F6
code readiness

Gate 2 rejects “looks plausible” designs when rows, ports, obligations, or trace links are missing.

Gate 3: do not reinvent code review, route it

Existing analyzers remain the final gate over emitted code.

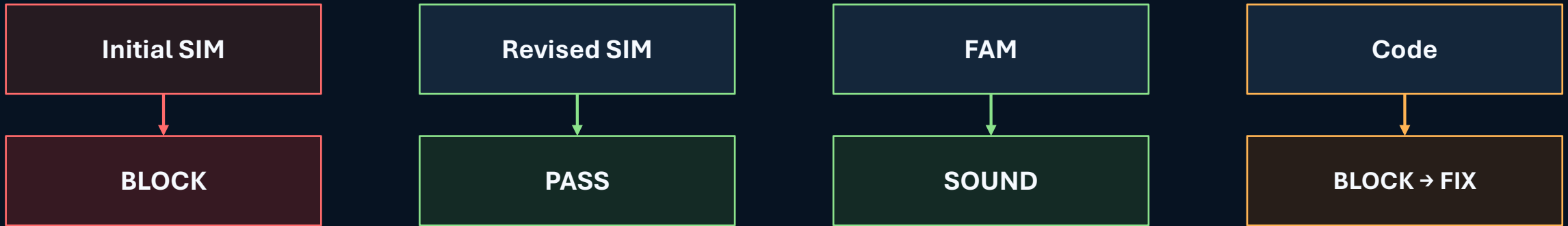


Key rule: the agent may propose, but cannot admit its own output.

Gate policy outside the agent's write path
Evidence stored and replayable
Probabilistic sensors may observe; deterministic policy decides

Worked example: payment worker

One small system exercises all three levels.



Gate 1 catches:

fire-and-forget payment
weak deduplication
unbounded restarts

Gate 3 catches:

unsafe shell call
pickle load
bare except
unbounded wait

Each gate sees a different class of failure.

Honest limits: this is an engineering hypothesis

The paper is explicit about what is built and what still needs measurement.

Claimed now

coherent assembly
prototype / reference run
explicit artifacts and gates

Not claimed

general correctness
universal architecture
validated productivity gain

To measure

blind comparison
false alarms
quality vs cost

A gate is useful only if it catches real omissions without crying wolf so often that teams disable it.

Takeaway: manage the constraints, not every diff

The developer's role moves upstream, but it must become infrastructure.

- 1 Intent must be visible before code exists
- 2 Architecture should be data, not only prose
- 3 Admission policy must be outside the agent's write path
- 4 Code review remains necessary, but it is the last gate

Agent proposes. Gate decides. Human owns authority.